AI-POWERED

SECURITY DEFENCE PLATFORM

Complete Software Development & Existing-Application Integration Plan

DOCUMENT CONTROL	DETAIL
Purpose	Convert the supplied Development Blueprint into an actionable engineering plan
Source baseline	AI-Powered Security Defence Platform — Development Blueprint, 29 Sep 2026 (15 pages)
Prepared for	Engineering leadership, cybersecurity, Android/backend/AI teams, QA, DevOps and product
Status	Proposed plan; repository, policies, datasets, and operational constraints require verification
Change-control rule	No implementation code or source changes before Phase 0 discovery and plan approval

CONFIDENTIAL • INTERNAL ENGINEERING WORKING DOCUMENT

This document distinguishes requirements from the source blueprint, engineering recommendations, externally verified platform facts, assumptions, and decisions requiring validation. It is not a repository audit, legal opinion, product certification, or benchmark report.

Document navigation and evidence labels

Label	Meaning
[DOCUMENT REQUIREMENT]	Explicitly stated in the supplied blueprint; preserve unless formally changed.
[VERIFIED FACT]	Supported by an identified official source checked for this plan.
[ENGINEERING RECOMMENDATION]	Proposed design intended to be maintainable, secure, and practical.
[ASSUMPTION]	Working assumption because repository, traffic, team, or deployment facts are unavailable.
[REQUIRES VALIDATION]	Must be tested, checked against current policy, or confirmed with stakeholders.
[MANAGEMENT DECISION]	Requires explicit product/security/legal/engineering approval.

Contents

• 1. Executive Summary

• 2. Requirements and Research Gap Analysis

• 3. Existing Repository Discovery and Compatibility

• 4. Target Architecture and Data Flows

• 5. Technology and Model Selection

• 6. Multi-Agent and Component Contracts

• 7. Feature-by-Feature Implementation

• 8. APIs, Data Models and Retention

• 9. Security, Privacy and Compliance

• 10. Phase-Wise Roadmap and Schedule

• 11. Testing and Model Evaluation

• 12. Infrastructure, Team and Budget

• 13. Risk Register and Decisions

• 14. Developer Implementation Checklist

• 15. What to Do First: First Sprint

• 16. Production Readiness Gates

• 17. References

1. Executive Summary

The source blueprint proposes extending an existing application into an AI-assisted defence platform covering phishing URLs, Android APK risk, social-engineering messages/transcripts, and supported call-risk screening. The key constraint is compatibility: preserve current user workflows and security functionality, measure baseline behaviour, and make each change reviewable and reversible.

[DOCUMENT REQUIREMENT] The design is a hybrid system, not a fleet of autonomous LLM agents. Deterministic security engines, threat-intelligence lookups, static analysis, calibrated ML where justified, and a bounded explanation layer feed a central workflow and evidence aggregator. AI output must not independently authorize security-sensitive actions.

[ENGINEERING RECOMMENDATION] Deliver a narrow, working vertical slice first: repository baseline → common evidence schema → URL analysis → report UI/API → regression and security tests. Harden the existing APK pipeline in parallel only after it has been traced and protected by tests. Message and call features follow after data-access, Android-policy, privacy, and dataset feasibility reviews.

Management decisions required

Decision	Recommended default	Approval needed from
Integration strategy	Extend current architecture; modular monolith initially; avoid broad refactor	Engineering lead
APK execution	Separate, disposable, constrained worker; never execute unknown APKs in API process	Security + DevOps
AI authority	Rules/validated engines own verdict; LLM only optional structured language analysis/explanation	Security + Product
Call protection	Prototype supported CallScreeningService and number reputation; no assumption of live audio	Android lead + Legal/Privacy
External providers	Use adapters; approve each provider's data retention, region, terms and cost before sending data	Security + Legal + Procurement
Release policy	Unknown/partial result when critical checks fail; no unsupported 'safe' verdict	Product + Security

Success definition

• Existing application workflows pass a documented regression suite before and after each change.

• Every assessment is traceable to evidence, timestamps, provider/source, and rule/model versions.

• Sensitive data is minimized, access-controlled, retained only under an approved policy, and deletion is tested.

• Critical dependencies failing results in partial/unknown status, not an unsupported clean verdict.

• Security and model performance are measured on representative, legally usable test corpora; no accuracy claims precede evaluation.

• Release has signed approval from engineering, product, security, and privacy/legal owners as applicable.

2. Requirements and Research Gap Analysis

The following traceability table maps the supplied blueprint's requirements into proposed implementation and acceptance evidence. IDs are planning IDs, not existing repository identifiers. References use the supplied document's section/page labels.

ID	Original requirement	Proposed implementation	Dependencies	Acceptance criteria	Reference
TR-01	Preserve existing functionality	Baseline tests; staged changes; explicit untouched paths	Repository, CI, current test environment	Existing critical workflows pass regression suite	§1, §13–15, pp.2,11–12
TR-02	Submit URLs for analysis	Validated URL intake; safe normalization; isolated fetcher; rules/reputation/model as justified	API boundary, SSRF-safe fetcher, reputation provider	Schema errors safe; blocked destinations never fetched; findings versioned	FR-01; §5A, §9
TR-03	Submit APKs for analysis	Quarantine, SHA-256, bounded parser, manifest/DEX/signature analysis; optional sandbox	Private storage, isolated worker, quotas	Malformed/oversized inputs rejected safely; job and report traceable	FR-02; §5B

ID	Original requirement	Proposed implementation	Dependencies	Acceptance criteria	Reference
TR-04	Analyze suspicious messages/transcripts	Authorized submission, redaction, structured indicator extraction, language evaluation	Consent, privacy filtering, dataset, optional model	Sensitive fields redacted as configured; explanation tied to findings	FR-03; §5C
TR-05	Call-risk assessment	Number reputation + supported screening role/API; user-submitted transcript separately	Android version/OEM, user role, Play policy, provider	Role denied/offline/unsupported cases fall back safely	FR-04; §5D
TR-06	Evidence-based risk report	Versioned evidence, risk, confidence, category, sources/timestamps and limitations	Common schema, risk policy, report API/UI	Report reproducible from recorded rule/model/source versions	FR-05; §8
TR-07	Feedback and incident reporting	Feedback endpoint, abuse controls, review queue; no direct model update	Review workflow, privacy policy	Feedback cannot modify production model/policy directly	FR-06; §9
TR-08	Authentication and authorization	Existing trusted identity; centralized policy; object-level ownership checks	Identity provider, API middleware	Cross-user object access tests denied for every endpoint	NFR-01; §11
TR-09	Reliability and resilience	Timeouts, idempotency, bounded retries, circuit breakers, partial result	Job store; optional queue	Duplicate requests do not create unsafe duplicate work; failures recover	NFR-02; §7
TR-10	Privacy and retention	Data inventory, redaction, consent, retention/deletion jobs	Legal basis, storage providers, processor terms	Deletion and access tests pass; no secrets/raw payload in logs	NFR-03; §10,12
TR-11	Observability	Structured redacted events, metrics, trace IDs, alerts	Current telemetry stack or selected equivalent	Critical failures and resource abuse are observable	NFR-04; §16
TR-12	Performance targets	Measure baseline; define per-service SLOs after workload test	Traffic assumptions, representative devices/data	Approved p50/p95/p99 and throughput targets measured in staging	NFR-05; §17
TR-13	Maintainable versioned contracts	Typed interfaces, schema/API versioning, reproducible artifacts	Current stack conventions	Contract tests and model/rule version stored per assessment	NFR-06; §6
TR-14	Safe untrusted-content processing	Constrained parsers, SSRF controls, output schema validation, allowlisted tools	Sandbox, egress controls, security tests	Prompt injection cannot invoke tools or alter policy; SSRF tests block internal targets	§2,4,5,7
TR-15	Threat-intelligence integration	Provider adapters with provenance, TTL, quotas, failure state	Approved providers and licensing	Provider outage/stale results represented explicitly	§4,5,7,18
TR-16	Cross-channel evidence correlation	Normalize and deduplicate related indicators only when authorized	Common schema, identity/consent scope	Correlated findings retain provenance and do not double-count evidence	§1,6,8
TR-17	Safe response and user guidance	Approved templates; confirmation before consequential action	Product policy, UI flows	No autonomous deletion/reporting/transaction blocking outside approved policy	§1,6,8
TR-18	Production deployment and recovery	Staged release, monitoring, backups, incident and rollback runbooks	CI/CD, operations, owners	Restore/rollback drill completed; release gates signed	§16,19

2.1 Explicit gaps in the source research

Gap	Why it matters	Required resolution
No repository/source code supplied	Cannot confirm stack, current features, paths, or tests	Phase 0 repository access and technical walkthrough
No representative labelled datasets	Cannot choose or claim model quality/thresholds	Dataset inventory, provenance/licensing review, label protocol and holdout set
No expected workload/SLO	Cannot size queue, worker pool, storage, or costs	Product traffic forecast and baseline/load tests
No provider/vendor decisions	Coverage, privacy, region, quotas and cost unknown	Threat-intel/LLM vendor evaluation and contract review
No retention/legal-basis decision	Sensitive APK/message/call data needs purpose and lifecycle controls	Data inventory and qualified India privacy/telecom counsel review
Call feature feasibility not proven on target devices	API/role/OEM/Play constraints may limit behaviour	Device matrix PoC and Play policy review
No approved score thresholds	False positives/negatives have different impacts	Security/product risk workshop and validation-based calibration
No deployment topology or incident ownership	Production readiness and on-call model unknown	Ops review, RACI and recovery exercise
No UI/UX requirements	Warnings can create confusion or unsafe actions	Prototype and usability review with safe-response principles

2.2 Assumptions that must not be treated as facts

• The current app may already contain a scanner, API, identity provider, dashboard, or storage. Each must be verified; no duplicate implementation should be created until discovery.

• The initial release can use a modular monolith for most orchestration if load and current architecture allow. APK parsing should still receive process/container isolation appropriate to the threat model.

• ML will be deferred unless a representative, legally usable dataset and evaluation harness exist.

• Any numerical performance target, budget, delivery date, accuracy rate, or model selection remains provisional until measured or quoted.

3. Existing Repository Discovery and Compatibility

3.1 Discovery rules

• Record repository URL/location, branch, commit hash, working-tree status, and uncommitted changes. Do not reset, clean, reformat, or upgrade dependencies.

• Map top-level directories, app entry points, build variants, Android modules, backend routers, middleware, worker/task modules, database models, storage, dashboard and deployment configuration.

• Inventory every current endpoint, authentication method, object-ownership check, external integration, permission, background task, and user-facing workflow.

• Run existing tests without unrelated fixes. Record command, environment, pass/fail, skipped tests, known warnings and baseline build artifacts.

• Trace the current APK scan end-to-end: discovery/upload, parsing, storage, verdict generation, deletion/quarantine, scheduled scanning, UI and error paths.

• Inspect manifests, Gradle/package manifests, lockfiles, secrets handling, network access, subprocess execution, file paths, exported Android components, deep links, WebViews and backup settings.

• Map data stores, schema migrations, retention/deletion jobs, logging fields, CI/CD, deployment/rollback and monitoring.

• Produce an exact file-by-file change plan with create/modify/untouched status, compatibility risks, tests and rollback before coding.

3.2 Repository inventory template

Area	Inspect	Evidence to collect
Android	settings/build scripts, app modules, manifest, permissions, min/target SDK, navigation, services	Build variants, permission rationale, API compatibility, exported components
Backend/API	entry points, routers, middleware, auth, schemas, config	Route inventory, authz matrix, error format, rate limiting
Scanner/workers	APK intake, parser calls, subprocesses, scheduled jobs	Data flow, resource limits, sandboxing, known failure modes

Area	Inspect	Evidence to collect
Persistence	models, migrations, repositories, object storage	Ownership fields, indexes, retention and deletion
AI/ML/TI	models, prompts, providers, API clients, rules/signatures	Versions, licenses, data sent externally, timeouts and fallback
UI/dashboard	screens, navigation, report models, error states	Current user journeys and backward compatibility
QA/CI/CD	unit/integration/device tests, workflows, signing and release config	Baseline result, artifact provenance, rollback capability
Operations	deployment manifests, secrets, telemetry, backup and incident docs	Environments, access boundaries, restore evidence

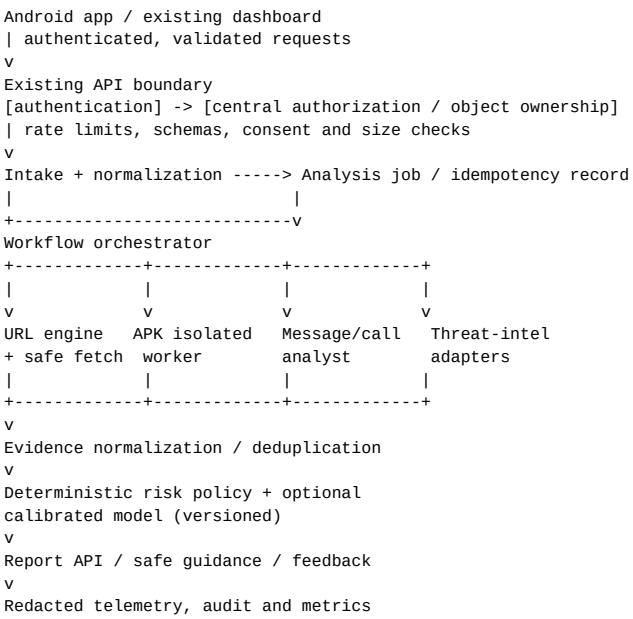
3.3 Baseline exit gate

No source changes until engineering reviews: (1) repository map, (2) baseline test report, (3) current APK scan flow, (4) security/data-flow notes, (5) exact change list, (6) regression plan, and (7) rollback plan. If tests cannot run, record the environmental blocker and agree a substitute baseline.

4. Target Architecture and Data Flows

[ENGINEERING RECOMMENDATION] Preserve current app/API/service boundaries where feasible. Start with a modular monolith or existing service layout, plus a separate isolated APK worker. Add a durable queue only when workload, latency, retry or isolation requirements justify it.

4.1 High-level architecture



4.2 Component responsibilities and trust boundaries

Component	Responsibilities	Trust/security boundary
Existing Android app/dashboard	Submission, consent UX, status, reports, feedback	Client is untrusted; no client-supplied role/verdict trusted
API boundary	Authn, authz, schema validation, quotas, idempotency, object ownership	Every object read/write re-checks owner/tenant
Intake/normalization	Canonical event type, payload limits, safe normalization	Preserve original evidence safely; reject unsupported forms
Orchestrator	Routes only approved tasks; deadlines, cancellation, bounded concurrency	No arbitrary tool execution; explicit allowlists
URL engine/fetcher	URL features, reputation, redirects, safe metadata	Dedicated egress restrictions; block internal/private/metadata destinations
APK worker	Bounded parsing, signature/manifest/DEX/native indicators	Disposable, non-root, no production secrets, no host mounts, default-deny egress
Message/call analyst	Extract contextual signals from authorized inputs	Redact; treat text as data; optional model cannot perform actions

Component	Responsibilities	Trust/security boundary
Threat-intel adapters	Provider queries, TTL, source/time/provenance	Allowlisted endpoints; handle stale, missing and poisoned data
Evidence/risk engine	Normalize, deduplicate, score, classify, mark unknown	Versioned deterministic policy; no LLM authority over verdict
Storage	Metadata, reports, quarantine binaries, audit records	Private, encrypted, least privilege, lifecycle deletion
Observability	Metrics, trace IDs, structured security events	Redact payloads and secrets; restricted audit access

4.3 Component-level sequence

```
Client -> API: submit event + idempotency key
API -> Identity/PDP: authenticate and authorize resource/action
API -> Validator: size/type/schema/consent/rate checks
API -> Job store: persist job and opaque payload reference
Orchestrator -> approved engines: run bounded checks in parallel where safe
Engines -> Orchestrator: typed findings + provenance + status
Orchestrator -> Evidence engine: normalize, deduplicate, identify missing checks
Evidence engine -> Policy: versioned score/category/confidence/unknown state
Guidance layer -> Report API: approved explanation based only on findings
Report API -> Client: authorized status/report
Telemetry -> Audit/metrics: minimized, redacted operational evidence
```

4.4 Feature data flows

URL/phishing

```
Submit -> validate scheme/length -> normalize hostname/IDN safely -> extract lexical/domain features -> reputation lookup -> controlled safe fetch
```

Critical controls: do not fetch URLs inside the API process; block loopback, private, link-local and cloud metadata destinations; revalidate redirects/resolution; bound response bytes, content types and time; do not execute downloaded scripts. HTTPS is not proof of benignness.

APK

```
Discover/upload -> type/size/quota checks -> private quarantine -> SHA-256 -> bounded metadata/manifest/DEX/native/signature parsing -> reputation
```

Critical controls: compressed and expanded size, entry count, nesting, path traversal, CPU, memory and wall-clock limits. Never run untrusted APKs in production API context or on the user's primary device.

Message/social engineering

```
Explicit user submission / approved integration -> consent and scope check -> local redaction -> structured indicators (claimed authority, OTP/PIN
```

Call-risk

```
CallScreeningService role/device support -> local/approved number reputation + caller metadata -> bounded decision within platform deadline -> all
```

[VERIFIED FACT] Android's CallScreeningService supports call screening and caller identification through the user-selected service; incoming screening must respond within five seconds. It is not a general live-call-audio transcription interface. Verify target OS/OEM behaviour and role UX. Source: Android Developers API reference. (https://developer.android.com/reference/android/telecom/CallScreeningService)

4.5 Failure and recovery behaviour

Failure	Required behaviour
Reputation provider timeout	Record provider unavailable; continue with available checks; mark partial/unknown if material
Model unavailable or malformed output	Use validated deterministic fallback; never convert failure into 'safe'
Queue outage	Return clear temporary failure or use bounded in-process path only if designed; avoid unbounded synchronous work
Worker crash / archive bomb	Terminate worker within resource budget, record sanitized failure, quarantine/reject, alert if repeated
Duplicate submission	Idempotency and job deduplication; do not duplicate external cost/actions
Authorization failure	Deny without disclosing whether another user's object exists
Retention deletion failure	Retry with bounded policy, alert, audit unresolved data; do not silently claim deletion
Guidance model hallucination	Reject schema/unsupported claims; use approved templates based on validated findings

5. Technology and Model Selection

Technology selection remains provisional until Phase 0 identifies the current stack. The default is to reuse working dependencies and conventions rather than rewrite the platform. The options below are evaluation candidates, not claims that the repository uses them.

5.1 Architecture and infrastructure options

Decision	Option A	Option B	Provisional direction and trade-off
Backend shape	Modular monolith / existing service	Multiple independently deployed services	Start modular; split only for isolation, independent scale or deployment needs
Async work	Bounded background task/in-process worker	Durable queue + worker pool	Start with existing task mechanism; queue when retries/load/job duration justify operations
Relational state	Existing database	PostgreSQL if compatible/new need proven	Reuse existing persistence; PostgreSQL is a candidate, not mandatory
Binary storage	Private filesystem/object storage already available	Managed object storage with lifecycle rules	Use private quarantine with random IDs, short-lived access, encryption and deletion
APK isolation	Restricted process/container	Disposable VM or dedicated sandbox	Container may suffice for static parsing after threat review; dynamic execution merits stronger isolation
Android	Existing Kotlin/Java app structure	New framework or rewrite	Keep current framework; use native platform APIs where required
API contracts	Current schema/validation conventions	OpenAPI-first contracts	Reuse conventions and add contract tests; document/version public interfaces
Observability	Existing logs/metrics stack	OpenTelemetry-compatible instrumentation and new backend	Instrument existing stack first; avoid duplicate observability systems

5.2 Detection approaches

Use case	Candidate approach	Recommended starting point	Trade-offs / validation
URL/phishing	Rules + reputation; lexical ML; page-content classifier	Rules/reputation + safe URL features first	Fast and explainable; provider coverage/staleness; ML requires representative time-split dataset
APK malware	Hash/reputation; manifest/DEX/signature heuristics; static ML; dynamic sandbox	Hash/reputation + bounded static analysis	Static features can miss obfuscation/novel malware; dynamic sandbox adds cost and risk
Message/scam	Rules and structured indicators; TF-IDF/linear model; compact transformer; LLM	Rules + typed indicator extraction; benchmark a small classifier	Multilingual/code-switching, class imbalance and context can shift results
Call reputation	Local/provider number reputation; supported caller ID/screening	Reputation adapter + opt-in CallScreeningService PoC	Numbers can be spoofed/reassigned; policy and OS role constraints
Transcript classification	Rules + evaluated multilingual text model; optional LLM structured extraction	Redaction + structured rules, then compare models	Speech-recognition errors and language imbalance require separate evaluation
Explanation	Approved templates; optional LLM summarizer	Templates first; optional constrained LLM after privacy review	LLM can hallucinate; explanations must be grounded in findings
Risk fusion	Versioned deterministic policy; calibrated classifier as secondary signal	Deterministic fusion with explicit unknown state	Avoid double-counting correlated evidence; thresholds require cost-based calibration

5.3 Model evaluation protocol

• Define each task and label taxonomy separately (URL, APK, message, transcript); do not reuse one generic 'fraud' label without justification.

• Document dataset source, collection date, label provenance, consent/legal basis, licensing, language/device/OS coverage and known contamination.

• Split by time and campaign/family where possible to reduce near-duplicate leakage; keep an untouched holdout set.

• Measure precision, recall, false-positive rate, false-negative rate, PR-AUC for imbalanced classes, calibration, latency, memory and throughput.

• Report subgroup performance for supported languages, code-switching, device classes and input quality; document unsupported slices.

• Choose thresholds using the operational cost of blocking legitimate callers/apps versus missing threats. Include an explicit insufficient-evidence state.

• Version model artifact, feature schema, training/evaluation code, threshold policy and release approval. Roll back if performance or drift gates fail.

5.4 ML/LLM decision gates

Gate	Evidence required	Decision
G0: Need	Rules/reputation gap demonstrated on target cases	If no material gap, do not add ML
G1: Data	Representative, legally usable labels; leakage review; train/validation/holdout plan	If insufficient, keep deterministic approach
G2: Baseline	Simple baseline (rules, lexical or linear model) measured	Only compare more complex model if it improves agreed metrics
G3: Deployment	Latency, memory, privacy, license and infrastructure checks	Choose on-device/server based on measured constraints
G4: Release	Threshold calibration, error review, model card, rollback plan	Human approval before staged release

5.5 Tooling candidates

Area	Candidates	Selection rule
Static APK analysis	Android SDK/aapt tools, JADX, apktool, MobSF as an analysis aid	Pin versions, review license and supply chain; run tools in restricted workers; validate findings
Backend security	Framework-native validation/auth middleware, dependency scanners, SAST, secret scanning	Prefer existing CI; pin versions and remediate high-risk findings
API security	OpenAPI schema/contract tests, integration tests, DAST in staging	Test object-level authorization and SSRF paths explicitly
ML lifecycle	scikit-learn baseline; PyTorch/Transformers only if justified; model registry/versioned artifacts	Compare against simple baseline; inspect model/data licenses and deployment footprint
Android QA	Gradle test tasks, emulator matrix, physical device/OEM matrix	Test target/min SDK and actual supported call-screening role
Observability	Existing telemetry; structured logs, metrics, trace IDs	Redaction by design; no secrets or full sensitive payloads in logs
CI/CD	Existing Git provider/workflows; signed build artifacts and staged deployment	Do not change release/signing pipeline without owner approval

Licensing, commercial use, model weights, dataset rights, vendor retention, region, pricing and service quotas are version- and contract-dependent. Verify each candidate from its official license/model card/provider terms at selection time; this plan does not assert a final license or price.

6. Multi-Agent and Component Contracts

The blueprint uses the term 'agent' for specialist components. Implement most as ordinary deterministic services or bounded workers with typed inputs/outputs. An LLM is optional only where language ambiguity or explanation provides measurable value.

Component	Input → output	LLM?	Permissions / controls	Failure handling and tests
Orchestrator	Validated event → approved task plan/status	No	Allowlisted routes, deadline, concurrency/token budgets; no arbitrary tools	Timeout/cancel; test duplicate jobs, partial failures and forbidden routes
URL engine	URL + safe metadata → indicators/findings	Usually no	Dedicated safe fetcher; egress blocklist/allowlist; response limits	Reject malformed URL; SSRF and redirect tests
APK worker	Quarantined APK → metadata/signature/manifest/DEX findings	No for core parsing	Non-root disposable worker; no secrets, host mounts or default egress	Fuzz malformed archives; kill on resource limit; worker crash tests
Message analyst	Authorized redacted message → structured signals	Optional	No external action; schema-constrained output; provider data-minimization	Prompt injection, multilingual, false-positive and redaction tests

Component	Input → output	LLM?	Permissions / controls	Failure handling and tests
Call-risk module	Call metadata/reputation or authorized transcript → risk signals	Optional for transcript only	Platform role/API boundaries; no implied audio access	Role denied, offline, 5-second deadline and fallback tests
Threat-intel adapter	Normalized indicator → source-attributed reputation result	No	Provider allowlist, credentials in secret store, quotas, TTL	Timeout, stale data, malformed response, poisoning and quota tests
Evidence/risk engine	Findings → risk/category/confidence/unknown	No	Versioned deterministic rules; evidence provenance; action separate from score	Correlated-signal tests, replay/reproducibility and calibration tests
Guidance generator	Sanitized findings → explanation/template	Optional	Approved templates; no tool execution or policy changes	Unsupported claim detection; schema validation and fallback
Feedback evaluator	Reviewed labels/corpus → evaluation report	No for release decision	Human approval; immutable model/version records	Abuse/rate tests; ensure feedback cannot auto-update production

6.1 Common finding contract

```
Finding (conceptual):
finding_id: UUID
category: string
indicator_type: enum/string
indicator_hash_or_redacted_value: string
severity: informational | low | medium | high | critical
confidence: number in [0,1] where defined by calibrated method
source: engine/provider identifier
observed_at: ISO-8601 timestamp
evidence_ref: opaque identifier
rule_or_model_version: string
limitations: list of strings
status: confirmed | suspected | unavailable | unsupported
```

6.2 Agent safety invariants

• Treat URLs, pages, APK strings, messages, transcripts, provider results and model outputs as untrusted data—not instructions.

• No specialist component receives unrestricted filesystem, network, shell, credentials or arbitrary tool access.

• Validate output against a strict schema; reject unknown fields, tool instructions, unbounded text and invalid scores.

• LLM output can propose a structured signal or explanation, but only the risk policy can issue the product verdict.

• User feedback is evidence for later review, not an automatic model or rules update.

7. Feature-by-Feature Implementation

7.1 URL and phishing detection

Step	Implementation	Verification
1 Intake	Validate URL length, scheme, encoding and supported formats; preserve submitted value in protected evidence	Malformed/unsupported scheme tests
2 Normalize	Parse host/path/query; handle IDNs explicitly; avoid lossy canonicalization	Unicode, punycode, mixed-case, encoded delimiters
3 Safe feature extraction	Domain age/reputation only when licensed; lexical features, known indicators, redirect metadata where safe	Deterministic feature tests
4 Optional fetch	Dedicated egress-restricted fetcher; block private/internal/metadata IPs; re-check DNS and every redirect	SSRF regression suite including DNS rebinding-style cases
5 Classification	Known-indicator/rules first; add ML only after dataset and baseline	Time split, precision/recall, false-positive review
6 Result	Findings, source/time, versions, confidence/limitations, guidance	Contract tests and replayability

7.2 APK analysis and quarantine

• Use the existing APK pipeline as the starting point; preserve current scan semantics until baseline and regression evidence exists.

• Validate size, MIME/signature, archive structure and quota before storage or parsing; assign opaque random object IDs and calculate SHA-256.

• Store binaries in private quarantine. Separate uploaded objects from public web serving and user-facing report data.

• Extract package metadata, signing certificate details, permissions, exported components, intent filters, suspicious API indicators, embedded URLs/domains and known hash matches.

• Treat permissions as contextual indicators, not proof. Record tool/parser version and findings provenance.

• Use a disposable non-root container/VM with read-only base, no host mounts, no production credentials, syscall/resource restrictions and default-deny egress. Dynamic execution is a separately approved capability.

• Define retention and deletion for original APK, extracted artifacts, derived metadata and reports; test deletion jobs.

7.3 Social-engineering and message analysis

• Initial supported path: explicit user submission or approved integration. Do not assume broad access to third-party messages.

• Extract structured indicators: claimed organisation, requested action, money/credential/OTP/PIN request, urgency, secrecy, remote-access request, URL/number, and inconsistencies.

• Use context and combinations; no single word such as 'urgent' is sufficient to label a scam.

• Redact OTPs, payment identifiers, phone numbers and unrelated personal content before external processing where feasible.

• Evaluate each language and code-switched slice independently; speech recognition errors require a separate transcript-quality evaluation.

7.4 Call-risk assessment

• Build two separate paths: (A) supported call screening and number reputation, (B) analysis of a transcript/audio file that the user is authorized to provide.

• Prototype Android CallScreeningService role request and response deadline on supported OS/OEM devices; cache/local reputation may be needed to meet latency.

• Treat caller ID as a claim, not proof of identity. Provide independent verification guidance such as calling a known official number.

• Do not design around hidden recording, live microphone capture or bypassing OS restrictions. If role/API unavailable, fall back to manual number lookup and user report.

• Google Play restricts SMS and Call Log permission groups and requires eligible core use cases and declarations/review. Recheck the current policy before implementation and submission. Official policy: https://support.google.com/googleplay/android-developer/answer/10208820

7.5 Evidence fusion and response

• Separate risk score, confidence, verdict, evidence and permitted action. Do not conflate model probability with a guaranteed risk percentage.

• Deduplicate correlated signals so multiple features from one URL/domain do not count as independent confirmations.

• Store source timestamps, rule/model/provider versions and unavailable checks. Make 'unknown/partial' an explicit result.

• Use approved templates for warnings; require confirmation for consequential actions. No autonomous deletion, third-party reporting or transaction blocking unless separately approved and implemented under explicit policy.

8. APIs, Data Models and Retention

8.1 Candidate API endpoints

These routes are design candidates from the source blueprint. Reuse existing routes and naming if equivalent functionality already exists.

Method / path	Purpose	Key controls
POST /api/v1/analyses/urls	Submit URL	Authz, scheme/length validation, SSRF-safe processing, quotas, idempotency
POST /api/v1/analyses/apks	Submit APK	Streaming limits, quarantine, content inspection, async job
POST /api/v1/analyses/messages	Submit message	Consent/scope, redaction, payload limit
POST /api/v1/analyses/call-events	Submit call metadata or authorized transcript	Explicit event type; no implied live-audio access
GET /api/v1/analyses/{id}	Read status/report	Object-level authorization on every request
POST /api/v1/reports/{id}/feedback	Submit correction	Rate limits, abuse controls, no direct model update
POST /api/v1/threat-reports	Report suspicious indicator	Provenance, deduplication, human review before trusted-feed promotion

8.2 Illustrative request/response

```
POST /api/v1/analyses/urls
Authorization: Bearer <token>
Idempotency-Key: <opaque-client-generated-key>
Content-Type: application/json
```

```
{
"url": "https://example.invalid/path",
"consent_context": {"purpose": "user_requested_url_check"},
"client_schema_version": 1
}
```

```
202 Accepted
{
"request_id": "opaque-id",
"analysis_id": "opaque-id",
"status": "queued",
"submitted_at": "ISO-8601",
"result": null,
"errors": []
}
```

```
GET /api/v1/analyses/{id}
{
"analysis_id": "opaque-id",
"status": "completed | partial | unknown | failed",
"risk_category": "informational | low | medium | high | critical | unknown",
"confidence": null,
"findings": [],
"checks_unavailable": [],
"policy_version": "version",
"limitations": []
}
```

Examples are conceptual and intentionally use placeholder identifiers. Final schemas, confidence semantics, error codes and auth conventions must follow the inspected framework and API style.

8.3 Core conceptual data models

Entity	Minimum fields	Important controls
AnalysisJob	analysis_id, owner/tenant, event_type, status, created/updated timestamps, idempotency reference, schema version	Object-level ownership; finite states; retry limits
SecurityEvent	event_id, event_type, submitted_at, source, subject_ref, consent_context, payload_ref, schema_version	Avoid raw payload in general event table; validate consent/scope
Finding	finding_id, category, indicator type, redacted/hash value, severity, confidence if defined, source, observed_at, evidence_ref, version	Provenance and versioning; do not log secrets
EvidenceReference	opaque ref, source type, content digest if justified, storage pointer, access scope, retention expiry	Private access; short-lived links; integrity controls
RiskAssessment	analysis_id, category, score if defined, confidence semantics, policy/model/rule versions, limitations, missing checks	Score and action are separate; reproducible decision
ThreatIntelResult	indicator type/value, provider, result, observed_at, expiry/TTL, request status	Provider provenance; cache public reputation only with TTL
Feedback	feedback_id, report_id, user/tenant, label, note-redacted, timestamp, review status	Rate limit; no direct production model update
AuditEvent	actor/service, action, resource ref, decision, timestamp, trace ID, minimal context	Append-only/controlled access where feasible; redact sensitive fields

8.4 Data lifecycle

```
Collect minimum -> validate/consent -> process in authorized boundary -> store findings separately from raw payload -> restrict access -> apply re
```

• Create a data inventory: purpose, legal basis/consent path, fields, access roles, storage region, processors, retention, deletion mechanism and incident response.

• Encrypt transport and storage; keep provider secrets server-side; do not place API keys in mobile binaries.

• Avoid retaining raw query strings, tracking tokens, full transcripts, APKs or personal identifiers unless required and approved.

• A hash of personal data does not automatically make it anonymous.

9. Security, Privacy and Compliance

9.1 Backend and API security

• Use the existing trusted identity integration if present; otherwise approve an authentication choice before implementation.

• Centralize authorization policy; default-deny unknown actions/resources; verify ownership/tenant on every read, update, report and delete.

• Use TLS, secret management, least-privilege service identities, key rotation, dependency pinning/scanning, secure configuration and redacted logs.

• Apply request and file limits, quotas, rate limits, safe errors, idempotency and bounded retries.

• Treat OWASP API Security Top 10 2023 as a baseline; specifically test broken object-level authorization, broken authentication, unrestricted resource consumption and SSRF. Official reference: https://api-security.owasp.org/editions/2023/en/0x11-t10/

9.2 Android security and policy

• Use only necessary permissions, request them at point of use with clear rationale, and provide graceful fallback on denial.

• Declare package visibility narrowly for actual use cases; do not imply inspection of every installed app/device state.

• Review exported activities/services/receivers, deep links, WebView configuration, backup behaviour, local data protection and release artifact.

• Call screening and live call audio are separate capabilities. Verify target OS/OEM, role opt-in, response deadline, privacy, local law and distribution policy.

• Recheck Google Play permission rules and declaration requirements immediately before implementation and release; policies can change.

9.3 India privacy and legal review

[REQUIRES VALIDATION] The source blueprint correctly treats India-specific data-protection applicability and commencement as a legal review item rather than a settled conclusion. Before release, qualified counsel should verify the current Digital Personal Data Protection framework, commencement notifications/rules, telecom/recording requirements, lawful basis, consent/notice, user rights, processor contracts, cross-border transfers and breach response for the actual product and data flows.

Question	Required evidence / owner
What data is collected and why?	Data inventory and purpose mapping — Product + Privacy
Is consent or another lawful basis required?	Written legal assessment per data flow — Counsel
Can data be sent to an LLM/TI provider?	Provider terms, retention/training/region/subprocessor review — Security + Legal
How long are APKs/transcripts retained?	Approved retention schedule and deletion tests — Product + Security
Can app request SMS/call-log permissions?	Current Play policy eligibility, declarations and Android alternatives — Android lead + Legal
What happens after a breach?	Incident response, notification duties and escalation owners — Security + Counsel

9.4 Threat model minimum

Threat	Control	Test evidence
SSRF via URL or redirect	Dedicated fetcher, destination checks after DNS/redirect, egress policy	Loopback/private/link-local/metadata/redirect test suite
Archive bomb/parser exploit	Bounded disposable worker, resource limits, no secrets/egress	Malformed archive fuzzing and timeout/crash tests
Cross-user report access	Central PDP/PEP and object ownership check	Two-user negative tests for all object endpoints
Prompt injection	Untrusted data separation, allowlisted tools, strict schema	Adversarial URL/message/transcript corpus
Provider poisoning/staleness	Provenance, TTL, corroboration and monitoring	Stale/malformed provider response tests
Sensitive-data leakage	Redaction, access control, encryption, retention/deletion	Log scanning, deletion verification, access audit
False positive blocks legitimate activity	Conservative policy, explainability, feedback and rollback	Reviewed false-positive cases and staged canary
Existing functionality regression	Baseline tests, small commits, explicit rollback	Before/after regression report

10. Phase-Wise Development Roadmap

Schedule is preliminary. It assumes a small cross-functional team, repository access at project start, an existing working application, no major rewrite, and no prolonged legal/provider procurement. Estimates are elapsed ranges for planning only; phases may overlap after dependencies are understood. They are not a delivery commitment.

Phase	Indicative range	Roles	Key deliverables	Exit gate	Rollback / risk
0. Discovery/baseline	1–2 weeks	Tech lead, Android, backend, QA	Repository map; baseline tests; APK flow; exact change plan	No source changes; baseline evidence reviewed	Stop if repository/build access unavailable
1. Requirements/threat model	1 week	Product, security, tech lead, privacy/legal	Requirements traceability; data inventory draft; threat model; architecture decisions	Owners approve scope, threat model and assumptions	Revisit scope if sensitive permissions/provider terms unclear
2. Security foundations	1–3 weeks	Backend, Android, security, QA	Authz boundaries, validation, redacted logging, configuration and contract tests	Default-deny and object-level authorization tests pass; regression remains green	Feature flag and rollback per change set
3. URL engine + evidence schema	2–4 weeks	Backend/security, QA, optional ML	URL intake, safe normalization/fetch boundary, TI adapter, report contract	SSRF suite passes; provenance/version fields present; unknown on failure	Disable remote fetch or provider independently
4. APK hardening	2–5 weeks	Security, backend, DevOps, QA	Quarantine, hashes, bounded static parser, worker isolation, retention	Archive bomb/path traversal/resource tests pass; existing scan regression passes	Feature flag worker; rollback to safe prior path only if not exposing risk
5. Message analysis	2–4 weeks	Backend, ML/NLP, privacy, QA	Authorized submission, redaction, structured signals, language test corpus	Consent/redaction tests pass; per-language metrics documented	Keep unsupported languages explicitly unsupported
6. Call feasibility/PoC	2–4 weeks	Android, security, product, legal	CallScreeningService role PoC, number reputation adapter, fallback UX	Supported device matrix and Play/legal review signed; no audio assumptions	Keep feature off if role/policy/device constraints fail
7. ML evaluation/integration	2–6 weeks; data-dependent	ML, security, QA, product	Baseline comparison, dataset report, calibration, model registry/versioning	Holdout metrics and error review meet pre-approved thresholds	Rollback model/rules to prior signed version
8. Orchestration/correlation	2–4 weeks	Backend, DevOps, security, QA	Bounded jobs, idempotency, timeouts, deduplication, partial results	Failure injection and retry/idempotency tests pass	Fallback to deterministic engine and unknown status
9. UI/report/feedback	2–4 weeks	Android/frontend, product, QA	Warnings, reports, limitations, feedback and safe response	Usability review, accessibility and regression checks pass	Feature flag UI; retain old workflows
10. System/security/performance QA	2–4 weeks	QA, security, DevOps, all engineers	Full regression, DAST, load, privacy/deletion, restore tests	No unresolved critical/high findings without formal risk acceptance	Release candidate rejected on gate failure
11. Staged deployment	1–2 weeks + observation	DevOps, security, product, support	Canary, dashboards, runbooks, rollback and incident exercise	SLOs met in staged rollout; owners approve production	Canary halt/rollback trigger documented

10.1 Dependencies and parallel work

• Phase 0 precedes all source changes. Phase 1 approvals precede security-sensitive design and permissions.

• Security foundations should be in place before public API expansion or external payload processing.

• URL work and APK worker hardening can overlap after repository discovery if owners are separate and shared evidence contracts are agreed.

• Message analysis can prototype in parallel with URL work only using explicitly authorized test data and an approved privacy boundary.

• Call feasibility research can run in parallel, but production implementation waits for Android role/device testing and policy/legal review.

• ML model integration depends on dataset provenance, baseline metrics, label agreement and evaluation infrastructure; model research can run earlier, production deployment cannot.

• Cross-channel correlation follows stable event/finding contracts; production launch follows end-to-end regression, security review and operational readiness.

10.2 Change control per phase

• One approved phase per change set; small, reviewable commits.

• Before edits: list exact paths, behavioural change, tests, and untouched areas.

• After edits: summarize diff, tests actually executed, failures/environment issues, security findings and rollback steps.

• Do not upgrade dependencies, reformat unrelated code, migrate data or add infrastructure without separate justification and approval.

• Verify working-tree state and expected runtime behaviour after each approved phase.

11. Testing and Model Evaluation

Test group	Minimum coverage	Pass evidence
Unit	URL normalization, feature extraction, parser bounds, scoring, deduplication	Repeatable test report
Integration/API	Schemas, authn/authz, ownership, idempotency, rate limits, error handling	Contract and negative authorization tests
Regression	Existing APK scan, scheduled scan, current routes, dashboard/navigation, report compatibility	Baseline vs candidate comparison
APK robustness	Malformed ZIP/APK, traversal, oversized expansion, entry/depth limits, worker timeout/crash	Worker remains isolated; no API crash or uncontrolled resource use
SSRF	Private/loopback/link-local/metadata IPs, redirects, alternate encodings, DNS changes	No forbidden destination reached in controlled test environment
Agent safety	Prompt injection, unexpected fields, tool instructions, oversized outputs, budget exhaustion	No tool/policy escalation; invalid output rejected
Call/Android	Role denied, unsupported OS/OEM, offline reputation, revoked permission, latency	Fallback works; platform deadline met on supported devices
Privacy	Consent, redaction, retention, deletion, logs, provider payload minimization	Deletion and leakage checks pass
ML	Precision/recall, FPR/FNR, PR-AUC, calibration, slices by language/device/time	Reproducible evaluation report with holdout data
Resilience	Provider timeout, queue outage, retry storm, duplicate submit, cancellation, partial result	Bounded recovery; unknown/partial rather than false-safe
Performance	p50/p95/p99, throughput, CPU/RAM/storage, worker concurrency, low-end Android	Measured against approved SLOs
Operations	Backup/restore, alerting, model rollback, deployment rollback, incident drill	Recorded exercise and owner sign-off

11.1 Acceptance thresholds

[REQUIRES VALIDATION] The source blueprint intentionally does not set numerical accuracy thresholds or service SLOs before representative data and workload baselines exist. Product and Security must approve task-specific thresholds before evaluating a release candidate.

• Use a holdout set untouched by feature/model tuning; prevent near-duplicate or campaign leakage across splits.

• Report confusion matrix and precision/recall at proposed operating thresholds; explain the cost of false positives and false negatives.

• For imbalanced datasets, include PR-AUC and calibration; accuracy alone is not sufficient.

• Set separate minimum criteria for each language/device/input-quality slice; unsupported slices must be identified.

• Track unknown/partial rate and provider coverage alongside detection metrics.

• Do not publish or present a model accuracy claim without a reproducible evaluation report.

11.2 Release gates

• No critical/high security finding remains unresolved unless a named owner documents explicit risk acceptance and compensating controls.

• No cross-user report, file, or evidence access succeeds in authorization tests.

• No untrusted APK executes in API process or production host context.

• No sensitive permission is requested before platform-policy and product-purpose approval.

• Critical dependency failure returns partial/unknown rather than an unsupported safe verdict.

• Model dataset, splits, metrics, thresholds, limitations and rollback are documented.

• Monitoring, backup/restore, retention deletion, incident response and rollback runbooks exist.

• Test reports describe commands actually run; environmental failures are not represented as passing tests.

12. Infrastructure, Team and Budget

12.1 Environment tiers

Environment	Minimum approach	Recommended controls / sizing method
Development	Developer laptops; Android SDK/IDE; repository and test fixtures	Use synthetic/sanitized data, pinned dependencies, local secrets excluded from Git; measure build/test requirements
CI/test	Isolated runners, unit/integration tests, Android emulator where appropriate	Ephemeral test credentials, artifact provenance, secret scanning, test containers
Staging	Production-like API/database/storage and worker isolation with non-production data	SSRF-safe test network, private quarantine, observability, synthetic datasets, access separation
Production	Existing approved deployment platform where suitable	Least privilege, managed secrets, encrypted storage, backups, alerting, staged rollout and recovery
Optional model training	CPU first; GPU only if measured model/training need warrants it	Estimate from dataset/model size and experiments; separate training data permissions
APK analysis	Dedicated restricted container/VM or sandbox pool	No production credentials, default-deny egress, disposable snapshots, CPU/RAM/disk/wall-clock limits

12.2 Minimum team roles

Role	Accountability	Indicative involvement
Product owner	Scope, UX, severity/action policy, acceptance and release decisions	Part-time throughout
Technical lead/architect	Repository discovery, integration plan, contracts, change control	High during Phase 0–2; review thereafter
Android engineer	Permissions, UI, device compatibility, call-screening PoC	High during discovery and mobile phases
Backend engineer	API, orchestration, persistence, TI adapters, reports	High during foundations and core engines
Security engineer/researcher	Threat model, URL/SSRF, APK isolation, security tests and incident controls	Continuous review
ML/NLP engineer	Dataset, baselines, evaluation, calibration and model lifecycle	Only when dataset/use case justifies ML
QA engineer	Regression, contracts, device matrix, performance and release evidence	Baseline onward
DevOps/SRE	CI/CD, secrets, isolated workers, monitoring, backup/restore, rollback	Foundations through production
Privacy/legal reviewer	Data-flow, consent, provider terms, India law and Play-policy review	Before sensitive-data features and release

12.3 Cost framework

A defensible numerical budget cannot be calculated without the current stack, region, traffic, provider choices, retention period, APK workload and team rates. Obtain current quotes/pricing from official vendor pages before approval. Track these cost categories:

• Engineering time: Android, backend, security, ML, QA, DevOps, product and legal review.

• Infrastructure: API compute, database, private object storage, backups, isolated APK workers/VMs, network egress and monitoring.

• External services: URL/domain reputation, malware/hash feeds, LLM inference, telephony-related data providers and vendor support.

• Model/data: dataset licensing/curation/label review, evaluation environments, optional GPU, model registry/artifact storage.

• Operations: on-call, incident response, log retention, security assessments, device lab and Play release maintenance.

Cost controls: run deterministic checks before expensive inference; cache only appropriate public reputation data with TTL/provenance; do not cache sensitive user payloads by default; enforce per-user quotas, concurrency caps, token budgets and provider spending alerts.

13. Risk Register and Decisions

ID	Risk	Severity	Mitigation	Owner	When
R-01	Existing feature regression	High	Baseline tests, small commits, explicit untouched paths, feature flags and rollback	Tech lead + QA	Before any code change
R-02	SSRF through URL fetch	Critical	Dedicated fetcher, DNS/redirect checks, egress restrictions, no ambient credentials	Security + Backend	Before URL fetch release
R-03	APK parser compromise/resource exhaustion	Critical	Disposable isolated worker, quotas, patching, no secrets/egress, fuzz tests	Security + DevOps	Before APK upload release
R-04	Cross-user report exposure	Critical	Central authz and object ownership on every endpoint; negative tests	Backend + Security	Before API release
R-05	False positives disrupt users	High	Evidence display, conservative thresholds, reviewed feedback, staged rollout	Product + Security	Before warning/block action
R-06	False negatives/novel threats	High	Layered detection, threat updates, unknown state, monitoring and evaluation	Security + ML	Ongoing
R-07	Call feature not available/policy eligible	High	Device/API PoC, policy/legal review, fallback; no live-audio assumption	Android + Legal	Before call feature scope lock
R-08	Sensitive-data leakage to providers/logs	High	Minimization, redaction, provider contract review, audit and deletion tests	Privacy + Security	Before external processing
R-09	LLM prompt injection/hallucination	High	No authority, allowlisted tools, strict schema, template fallback	AI + Security	Before LLM use
R-10	Threat-intel feed stale/poisoned	Medium/High	Provenance, timestamps, TTL, corroboration and provider health monitoring	Security	Before provider integration
R-11	Dataset licensing/label bias	High	Provenance/licensing review, label protocol, slice metrics, human review	ML + Legal	Before model release
R-12	Infrastructure cost/complexity grows	Medium	Modular start, cost budgets, measured service splitting and provider alerts	Tech lead + Finance	Architecture approval
R-13	India legal/policy interpretation incomplete	High	Qualified counsel review and release-time revalidation	Legal + Product	Before sensitive features / launch

ID	Risk	Severity	Mitigation	Owner	When
R-14	Retention deletion fails silently	High	Deletion jobs, verification metrics, retries, alerts and audit evidence	Backend + SRE	Before production

13.1 Decisions register

Decision needed	Default proposal	Approver	Deadline/gate
Which existing stack is authoritative?	Reuse existing supported framework/database/auth	Tech lead	Phase 0
Do URL checks fetch live page content?	No arbitrary fetch until safe-fetch boundary passes SSRF tests	Security	Before Phase 3
Which TI provider(s)?	Adapter abstraction; select after coverage/privacy/cost comparison	Security + Procurement	Before provider integration
Is ML necessary for MVP?	No until rules baseline and dataset justify it	Product + Security + ML	Phase 7
Call-screening scope?	Number reputation + supported opt-in role; no assumed audio access	Android + Legal + Product	Before Phase 6 release
Retention durations and legal basis?	Data inventory and counsel review; shortest justified retention	Privacy + Legal	Before data ingestion
Risk thresholds and actions?	Versioned conservative rules; unknown state; confirmation for consequential actions	Product + Security	Before release candidate
Production SLO/cost cap?	Set after baseline and load tests; alert budgets for providers	SRE + Product + Finance	Before production

14. Developer Implementation Checklist

Before implementation

- ■ Repository branch/commit/working-tree state recorded; no unapproved edits.
- ■ Current tests and build commands run; baseline report saved.
- ■ Current APK pipeline, app workflows, APIs, permissions, storage and external integrations traced.
- ■ Exact path-level change plan, regression tests and rollback plan reviewed.
- ■ Threat model, data inventory, trust boundaries and privacy questions assigned.

Security foundations

- ■ Authentication and object-level authorization enforced server-side.
- ■ Validation, size limits, quotas, rate limits and idempotency implemented using existing conventions.
- ■ Structured logs redact tokens, OTPs, transcripts, raw APKs and personal payloads.
- ■ Secrets remain server-side; TLS and least-privilege identities configured.
- ■ Every new endpoint has positive and negative authorization tests.

Detection modules

- ■ URL fetch is isolated; internal/private/metadata targets and unsafe redirects are blocked.
- ■ APK is quarantined, hashed, bounded and parsed in a constrained worker.
- ■ Message/transcript input is explicitly authorized and redacted.
- ■ Call features have platform role/API/policy validation and a no-audio fallback.
- ■ Evidence is versioned, source-attributed, timestamped and deduplicated.

AI/ML and orchestration

- ■ Deterministic baseline measured before ML adoption.
- ■ Dataset provenance, licensing, labels, splits and holdout are documented.

- ■ Model output schema, budgets, timeouts and failure behaviour are tested.
- ■ LLM cannot authorize actions or invoke arbitrary tools.
- ■ Unknown/partial outcomes and rollback are implemented.

Release

- ■ Regression, API/security, privacy/deletion, performance and resilience tests pass.
- ■ No unresolved critical/high security issues without signed risk acceptance.
- ■ Model evaluation and limitations reviewed; no unsupported accuracy claims.
- ■ Monitoring, backup/restore, incident runbooks and staged rollback are exercised.
- ■ Engineering, security, product and applicable legal/privacy owners sign release gate.

15. What to Do First: First Sprint

15.1 Information and access to obtain

- Read-only repository access first; write access only after discovery and approval.
- Build/run instructions, supported Android versions, signing/build variants and a safe test device/emulator.
- Access to current CI test reports, test accounts, staging API, deployment diagram and operational runbooks.
- Product walkthrough of current scan, report, deletion/quarantine, scheduled scan, login and dashboard flows.
- Inventory of current providers, API keys ownership (not secrets in chat), data stores, retention, logs and costs.
- Approved sample APKs and test corpus with clear permission/licensing; use benign and controlled malicious test artifacts only.
- Named owners for Android, backend, security, QA, DevOps, product and privacy/legal review.

15.2 First-sprint task board

Task	Owner	Dependency	Deliverable	Completion criteria
Kickoff, scope and access	Product + Tech lead	Sponsor approval	Scope and access record	Owners, objectives and non-goals agreed
Repository inventory	Tech lead + Android + Backend	Read-only repo access	Repository map and stack inventory	Entry points, APIs, modules, storage and deployment mapped
Baseline build/tests	QA + relevant engineers	Build environment	Baseline report	Commands/results and environmental failures recorded
Trace current APK workflow	Android + Backend + Security	Repository map	Current-state data-flow	Inputs, parser, storage, verdict, UI, scheduled tasks and failure paths documented
Security/data-flow review	Security + Privacy	Architecture/data inventory	Initial threat model and data inventory	Trust boundaries, sensitive fields, SSRF, authz, retention risks recorded
Requirements traceability review	Tech lead + Product + Security	Blueprint and repo findings	Approved/updated traceability matrix	Each requirement mapped to existing/new/gap with owner
Path-level change plan	Tech lead + QA + Security	Baseline and walkthrough	File-by-file plan, tests and rollback	Create/modify/untouched paths listed; no code changes yet
Management checkpoint	Sponsor + Engineering + Product	All above	Phase 0 approval record	Plan approved or specific discovery blockers assigned

15.3 Evidence to show management

Milestone	Evidence package
Discovery complete	Repository map, baseline test results, current APK flow, permissions/provider inventory, path-level change plan, rollback
Security foundation	Authorization matrix, schema/API contract tests, log-redaction checks, threat-model updates

Milestone	Evidence package
First URL prototype	Demo, test corpus and SSRF results, sample reports with provenance, regression comparison
APK hardening	Isolation design, resource-limit tests, malformed-file results, deletion evidence
ML gate	Dataset provenance, baseline comparison, confusion matrix, PR-AUC/calibration, slice analysis, model card and rollback
Release candidate	Full regression/security/privacy report, policy/legal sign-offs, operational runbooks, SLO/cost results, release checklist

16. Production Readiness Gates

Gate	Required evidence	Decision
G0 Scope	Requirements matrix; explicit unknowns and owners	No-go if critical scope ambiguity remains
G1 Compatibility	Baseline regression and approved file-level plan	No-go if existing critical behaviour cannot be tested or restored
G2 Security foundations	Authz tests, secrets/log review, validation and rate limits	No-go on cross-user access or critical boundary weakness
G3 Safe analysis	SSRF suite; APK isolation/resource limits; failure handling	No-go if untrusted content can access internal systems or production secrets
G4 Privacy/platform	Data inventory, retention/deletion, provider terms, Android/Play/legal review	No-go for unapproved sensitive collection or provider transfer
G5 Detection quality	Reproducible evaluation, thresholds, limitations, unknown state	No-go if quality criteria not agreed/met or unsupported claims are used
G6 Operations	Monitoring, alerting, backup/restore, rollback and incident exercise	No-go if recovery owner/runbook is missing
G7 Approval	Engineering + security + product + applicable legal/privacy sign-off	Release only after recorded approval

Final recommendation

Treat this as a controlled extension of the existing product, not a greenfield rewrite. The first deliverable is not code: it is a verified repository baseline and approved implementation plan. Then build a small end-to-end URL/evidence/report slice, harden APK processing under isolation, and introduce message/call/ML capabilities only when their data, platform and evaluation prerequisites are satisfied. Preserve unknown/partial states, version every decision input, and make rollback a first-class release feature.

17. References and Source Notes

The architecture and requirement baseline is the supplied 15-page document, *AI-Powered Security Defence Platform — Development Blueprint*, dated 29 September 2026. It states that it is a proposal, not a repository audit, legal opinion, product certification or benchmark.

ID	Source	URL / verification path	Use in plan
R1	Android Developers — CallScreeningService API reference	https://developer.android.com/reference/android/telecom/CallScreeningService	Supports screening/caller-identification role and the five-second response constraint; does not provide general live-audio transcription.
R2	Google Play Console Help — Use of SMS or Call Log permission groups	https://support.google.com/googleplay/android-developer/answer/10208820	Sensitive SMS/Call Log permissions are restricted; eligible use cases, role requirements and declarations/review must be checked.
R3	Google Play Console Help — Permissions and APIs that Access Sensitive Information	https://support.google.com/googleplay/android-developer/answer/16558241	Policy summary for sensitive information permissions.
R4	Android Developers — Improve your app's security	https://developer.android.com/privacy-and-security/security-best-practices	Android security baseline for storage, communication and app design.

ID	Source	URL / verification path	Use in plan
R5	Android Developers — Permissions updates in Android 11	https://developer.android.com/about/versions/11/privacy/permissions	Permission behaviour and privacy changes; verify against target SDK versions.
R6	Android Developers — Package visibility in Android 11	https://developer.android.com/about/versions/11/privacy/package-visibility	Package visibility restrictions and declarations.
R7	OWASP API Security Top 10 2023	https://api-security.owasp.org/editions/2023/en/0x11-t10/	API risks including broken object-level authorization and broken authentication.
R8	OWASP Mobile Top 10	https://owasp.org/projects/mobile-top-10	Mobile security risk baseline; not a substitute for app-specific threat modelling.
R9	India DPDP framework	Verify current official MeitY/Gazette notifications and commencement/rules at implementation and release	Legal applicability and commencement must be confirmed by qualified counsel; this report does not make a legal determination.

Evidence limitations

• No repository or source-code files were included with the supplied research blueprint; no repository-specific paths or current capabilities are asserted.

• No tests were executed for the product, no model benchmark was run, and no production cost was quoted.

• Official platform pages and policies may change. Recheck the exact policy text, effective dates, target Android versions and jurisdictional applicability before implementation and release.

• Numerical thresholds, model choices, SLOs, budget and schedule remain provisional until measured or confirmed.

End of document.